Plan de corrección

Cliente: Empresa Ejemplo

Resumen

Arreglos rápidos que hace el kit: 1 · Para un desarrollador: 13 (≈5.4 h) · Solo el cliente puede hacerlos: 6

1. Urgente (críticos)

Llaves y contraseñas expuestas (4)

Quién: Cliente · **Tiempo aprox.:** 1.3 h

Llave	Dónde	Dónde revocarla
Llave de acceso de AWS	.env:1	https://console.aws.amazon.com/iam/home#/security_credentials
URL de base de datos con contraseña	.env:2	Panel de la base de datos: cambiar la contraseña de ese usuario
Llave secreta de Stripe en producción	billing/stripe.js:1	https://dashboard.stripe.com/apikeys
Llave service_role de Supabase (acceso total a la base de datos)	src/lib/supabase.ts:1	https://supabase.com/dashboard/project/_/settings/api

1. Entrar al proveedor de esta llave (columna "Dónde revocarla") y **revocarla** (borrarla o desactivarla). Hasta que se revoque, sigue sirviendo aunque se borre del código.
2. Crear una llave nueva y guardarla en las variables de entorno del servidor o hosting, o en un gestor de secretos. Nunca en un archivo del proyecto.
3. Cambiar el código para leerla de la variable de entorno (por ejemplo `process.env.NOMBRE` o `os.environ["NOMBRE"]`).
4. Si estaba en el historial de git: borrarla del historial con `git filter-repo` o pedir ayuda al desarrollador. Si el repositorio fue público alguna vez, darla por comprometida aunque ya se haya revocado.
5. Revisar en el proveedor si hubo uso extraño de la llave (cobros, accesos) desde que se subió.

Las reglas de Firebase permiten acceso a cualquiera (if true)

Dónde: `firestore.rules:4` · **Quién:** Desarrollador · **Tiempo aprox.:** 45 min

1. Cambiar las reglas por las del archivo `firebase-rules-ejemplo.txt` del kit, adaptadas a sus colecciones.
2. Publicar las reglas y probar sin sesión iniciada.

2. Inmediato (altos)

Llaves y contraseñas expuestas (2)

Quién: Cliente · **Tiempo aprox.:** 40 min

Llave	Dónde	Dónde revocarla
Llave de API de Anthropic	.mcp.json:10	https://console.anthropic.com/settings/keys
Llave de API de OpenAI	config.py (commit de94084f)	https://platform.openai.com/api-keys

1. Entrar al proveedor de esta llave (columna "Dónde revocarla") y **revocarla** (borrarla o desactivarla). Hasta que se revoque, sigue sirviendo aunque se borre del código.
2. Crear una llave nueva y guardarla en las variables de entorno del servidor o hosting, o en un gestor de secretos. Nunca en un archivo del proyecto.
3. Cambiar el código para leerla de la variable de entorno (por ejemplo `process.env.NOMBRE` o `os.environ["NOMBRE"]`).
4. Si estaba en el historial de git: borrarla del historial con `git filter-repo` o pedir ayuda al desarrollador. Si el repositorio fue público alguna vez, darla por comprometida aunque ya se haya revocado.
5. Revisar en el proveedor si hubo uso extraño de la llave (cobros, accesos) desde que se subió.

Claude Code configurado para no pedir ningún permiso

Dónde: `.claude/settings.json` · **Quién:** Desarrollador · **Tiempo aprox.:** 10 min

1. En `.claude/settings.json`, quitar `"defaultMode": "bypassPermissions"`.
2. Permitir solo comandos concretos, como `Bash(npm test)`.

Caracteres ocultos en un archivo de instrucciones para IA (3)

Dónde: `CLAUDE.md:2` · **Quién:** Desarrollador · **Tiempo aprox.:** 20 min

1. Abrir el archivo, borrar los caracteres invisibles y revisar qué instrucciones escondían.
2. Averiguar en el historial de git quién los agregó.

SQL armado uniendo texto (posible inyección SQL)

Dónde: `api/app.py:3` · **Quién:** Desarrollador · **Tiempo aprox.:** 30 min

1. Cambiar la consulta para pasar los valores como parámetros (`$1`, `?`, `%s`) en vez de unir texto.

Secreto expuesto al navegador mediante NEXT_PUBLIC_SUPABASE_SERVICE_ROLE_KEY

Dónde: `src/lib/supabase.ts:2` · **Quién:** Desarrollador · **Tiempo aprox.:** 1 h

1. Quitar el prefijo público (`NEXT_PUBLIC_`/`VITE_`) de esa variable.
2. Mover el uso de ese secreto a una ruta o función del servidor.
3. Rotar la llave, porque ya se publicó en el sitio.

La tabla 'customers' no tiene seguridad por fila (RLS)

Dónde: `supabase/migrations/001_init.sql:1` · **Quién:** Desarrollador · **Tiempo aprox.:** 45 min

1. Revisar y correr el archivo `supabase-rls.sql` del kit en el editor SQL de Supabase (activa la seguridad por fila).
2. Ajustar las políticas para que cada usuario vea solo sus registros.
3. Probar la app sin sesión iniciada: no debe poder leer datos.

Política de base de datos que permite todo (using true)

Dónde: `supabase/migrations/001_init.sql:4` · **Quién:** Desarrollador · **Tiempo aprox.:** 45 min

1. Cambiar `using (true)` por una condición de dueño, por ejemplo `using (auth.uid() = user_id)`.
2. Probar sin sesión iniciada y con otro usuario.

Archivo de entorno (.env) subido al repositorio

Dónde: `.env` · **Quién:** Lo hace el kit · **Tiempo aprox.:** 10 min

1. El kit agrega `.env` a `.gitignore` y crea `.env.example` solo con los nombres de las variables (sin valores).

2. Quitar el archivo del repositorio sin borrarlo de la computadora: `git rm --cached .env` y hacer commit.
3. Rotar los valores que tenía (ver el paso de llaves) porque siguen en el historial.

3. Esta semana (medios)

Claude Code con permiso previo para comandos de terminal amplios

Dónde: `.claude/settings.json` · **Quién:** Desarrollador · **Tiempo aprox.:** 10 min

1. En `.claude/settings.json`, cambiar reglas como `Bash(*)` por los comandos exactos que el proyecto usa.

Todos los servidores MCP del proyecto se aprueban automáticamente

Dónde: `.claude/settings.json` · **Quién:** Desarrollador · **Tiempo aprox.:** 5 min

1. Quitar `enableAllProjectMcpServers` y aprobar los servidores MCP por nombre.

Verificación de certificados TLS desactivada

Dónde: `api/app.py:4` · **Quién:** Desarrollador · **Tiempo aprox.:** 30 min

1. Descargar el certificado (CA) del proveedor de la base de datos o servicio.
2. Configurar la conexión para usarlo y verificar, por ejemplo en Node `ssl: { ca: fs.readFileSync('ca.pem'), rejectUnauthorized: true }`.
3. Probar la conexión.

Modo debug activado

Dónde: `api/app.py:5` · **Quién:** Desarrollador · **Tiempo aprox.:** 10 min

1. Leer el modo debug de una variable de entorno y dejarlo apagado en producción.

4. Este mes (bajos)

El servidor MCP 'crm' usa un paquete sin versión fija

Dónde: `.mcp.json` · **Quién:** Desarrollador · **Tiempo aprox.:** 10 min

1. Fijar la versión exacta del paquete del servidor MCP (`paquete@1.2.3`).

Sin archivo de bloqueo (lockfile) para dependencias de JavaScript

Dónde: `package.json` · **Quién:** Desarrollador · **Tiempo aprox.:** 5 min

1. Correr `npm install` (o `pnpm install` / `yarn`) y subir el archivo de bloqueo que se genera.

5. Archivos listos en este kit

Cada carpeta tiene los archivos para ese proyecto. **Aplicar arreglos rápidos** copia al proyecto las reglas para IA (`AGENTS.md`, `CLAUDE.md`), `.env.example` y el bloqueo de llaves (`.pre-commit-config.yaml`), y agrega líneas a `.gitignore`. Nunca sobrescribe ni cambia código. El flujo de GitHub y Dependabot los instala el auditor, porque cambian cómo funciona el repositorio.

- **tienda-en-linea** (`tienda-en-linea/`): `.env.example`, `.github/dependabot.yml`, `.github/workflows/guardmycode.yml`, `.gitignore-additions.txt`, `.pre-commit-config.yaml`, `AGENTS.md`, `CLAUDE.md`, `firebase-rules-ejemplo.txt`, `fix-with-claude.md`, `supabase-rls.sql`

6. Protección permanente (plan mensual)

- **Bloqueo de llaves antes de cada commit:** `.pre-commit-config.yaml` con Gitleaks. Cada desarrollador corre una vez `pip install pre-commit` y `pre-commit install`.
- **Revisión en cada cambio:** `.github/workflows/guardmycode.yml` corre este escáner en cada pull request y bloquea problemas altos.
- **Librerías vulnerables:** `.github/dependabot.yml` propone actualizaciones de seguridad automáticamente.
- **Reglas para la IA:** `AGENTS.md` (y `CLAUDE.md`) le dicen a Claude, Cursor, Copilot y otros qué nunca deben hacer.
- **En GitHub:** Settings → Code security → activar *Secret scanning* y *Push protection* donde el plan lo permita, y exigir revisión antes de unir cambios a main.
- **Re-escaneo mensual** con GuardMyCode y reporte de una página para la dirección.

7. Para el desarrollador: corregir con Claude Code

Abrir el proyecto en Claude Code (o Cursor) y pegar el contenido de `fix-with-claude.md` de la carpeta del proyecto. Revisar cada cambio antes de publicarlo.